

Generic Sudoku as a Constraint Satisfaction Problem

Computational Logic – Practical Assignment

Samir Mansour

BSc in Computer Science, University of Minho

October 5, 2026

Abstract

This report describes the modelling and solving of an $n^2 \times n^2$ Sudoku as a constraint satisfaction problem (CSP). The central idea is a single abstraction, the *cell group* (**box**), from which rows, columns, blocks, random clues and several Sudoku variants are obtained simply as different groups, without changing the CSP model. Solving is done with the CP-SAT solver from OR-Tools. The code was validated automatically for $n = 2$ and $n = 3$ and tested up to $n = 6$ (36×36 grid).

Contents

1	Introduction	2
2	Theoretical background and choice of technique	2
2.1	The alldifferent constraint	2
2.2	Why CP and not linear integer programming	2
3	Mathematical model	3
3.1	The groups of classic Sudoku	3
3.2	Note on the clues	3
4	Implementation	3
4.1	Requirements and code	3
4.2	Architecture	3
4.3	box : the generic group (R1)	3
4.4	cube and path (R2, R3)	4
4.5	Random clues (R4)	5
4.6	CSP model and solving (R5)	5
4.7	Full Sudoku (R6)	6
5	Validation	6
6	Results	6
6.1	Random puzzles with no solution	6
6.2	Scale	7
7	Extensions	7
8	Discussion and limitations	7
9	Conclusion	8

1 Introduction

A classic Sudoku is an $n^2 \times n^2$ grid (typically $n = 3$) in which every row, every column and every $n \times n$ block contains all the values from 1 to n^2 without repetition. It is a canonical CSP example: the rule is always the same (*a set of cells must hold pairwise different values*), and what changes from row to row, column to column and block to block is only *which cells belong to the set*.

The goal of the assignment is to build a Sudoku generator and solver, parameterised by n , that:

- R1.** represents any group of cells with the “all different” constraint in a generic class (**box**);
- R2.** builds blocks and rows/columns as specialisations of that class (**cube** and **path**);
- R3.** generates random initial clues using the same abstraction;
- R4.** assembles a CSP model that accepts an arbitrary number of groups, and solves it;
- R5.** puts everything together into a full Sudoku and validates it automatically.

The rest of the report is organised as follows: section 2 connects the problem to the course material and justifies the chosen technique; section 3 presents the mathematical model; section 4 describes the implementation; sections 5 and 6 present validation and results; section 7 covers the extensions; and section 8 discusses decisions and limitations.

2 Theoretical background and choice of technique

2.1 The alldifferent constraint

Let S be a subset of the variables of a problem. The constraint $\text{alldifferent}(S)$ is satisfied by an assignment σ when, for any two distinct variables $x, y \in S$, we have $\sigma(x) \neq \sigma(y)$. This is written

$$\text{alldifferent}(S) \equiv \bigwedge_{x \in S} \bigwedge_{y < x} x \neq y.$$

2.2 Why CP and not linear integer programming

In linear integer arithmetic (LIA) the predicates have the form $c_1x_1 + \dots + c_nx_n \leq c_0$. The relation $x \neq y$ is not one of these predicates: it is equivalent to $x < y \vee x > y$, a disjunction, and so it has no direct representation in an integer programming model. According to chapter 2, a discrete problem that goes beyond the descriptive power of LIA is a *Constraint Programming* (CP) problem.

There are two ways to solve Sudoku with the course material:

CP model (chosen). One integer variable $x_{i,j} \in [1, N]$ per cell and one alldifferent constraint per group. The CP-SAT solver supports alldifferent directly and, as chapter 2 notes, uses strategies specific to this constraint to simplify the search tree while it is being built. Those strategies do not apply to generic constraints.

BIP model (alternative, not followed). Binary variables $X_{i,j,v} = 1$ iff cell (i, j) holds value v . The constraints “each cell has exactly one value” and “each value occurs exactly once in each group” are equations $A \cdot Y = 1$, that is, a *set partition* problem. It is linear and solvable with SCIP, but it uses N^3 variables and loses the direct correspondence between “group of cells” and “constraint”.

CP-SAT is also the course’s suggestion. In terms of decidability there is no problem: the domain is finite, so the problem is decidable (the search always terminates) and, in the worst case, has exponential complexity, like SAT.

3 Mathematical model

Let $N = n^2$. The model has one integer variable per cell,

$$x_{i,j} \in [1, N], \quad i, j \in \{0, \dots, N-1\},$$

read as “ $x_{i,j}$ is the value of cell (i, j) ”. This matrix $X \in [1, N]^{N \times N}$ is the *allocation matrix* of the problem, as in the allocation problems studied in the course.

A **group** g is a pair (C_g, v_g) made of a set of cells $C_g \subseteq \{0..N-1\}^2$ and a partial function $v_g : C_g \rightarrow [1, N]$ saying which cells are pinned and to what value. Each group contributes to the model with

$$\text{alldifferent}(\{x_{i,j} \mid (i, j) \in C_g\}) \tag{1}$$

$$\forall (i, j) \in \text{dom}(v_g) \cdot x_{i,j} = v_g(i, j) \tag{2}$$

3.1 The groups of classic Sudoku

- **Row** r : $C = \{(r, j) \mid j < N\}$.
- **Column** c : $C = \{(i, c) \mid i < N\}$.
- **Block** (a, b) , with $0 \leq a, b < n$: $C = \{(a n + p, b n + q) \mid p, q < n\}$.
- **Clues**: a group with k distinct cells, each pinned to a value $v \in [1, N]$.

A full Sudoku is the conjunction of the constraints of all groups, which gives $3N$ alldifferent constraints over N variables each.

3.2 Note on the clues

Random clues may repeat values: two clues in different rows and columns may both be 5. So, for the clue group, constraint (1) is *wrong* and only constraint (2) should be applied. The model therefore has two methods: one that applies (1) and (2), and another that applies only (2). It remains true that the model does not distinguish the *origin* of the groups; it is only told *how* to use each one.

4 Implementation

The code was written in Python 3 with the OR-Tools library (CP-SAT), in a Jupyter notebook (`sudoku_csp_ans_en.ipynb`) with the same structure as the course worksheets: problem analysis, implementation in numbered exercises, validation and extensions.

4.1 Requirements and code

4.2 Architecture

4.3 box: the generic group (R1)

The class stores a dictionary $(i, j) \mapsto \text{value or None}$ (**None** is a free cell). It knows nothing about rows, columns or blocks. The parameter **n** is used only to validate inputs: **add** raises **IndexError** for coordinates outside the grid and **ValueError** for values outside $[1, N]$. Adding a cell that already exists, without a value, does not erase the previous value; two different values for the same cell are rejected.

Requirement	Code	Exercise
R1	<code>box (add, matrix, pinned)</code>	1
R2	<code>cube(n, i, j)</code>	2
R3	<code>path(n, start, end)</code>	2
R4	<code>random_clues(n, k)</code>	3
R5	<code>sudoku_model (add_groups, add_clues, solve)</code>	4
R6	<code>full_sudoku, generate_and_solve</code>	5

Table 1: Correspondence between the assignment requirements and the code.

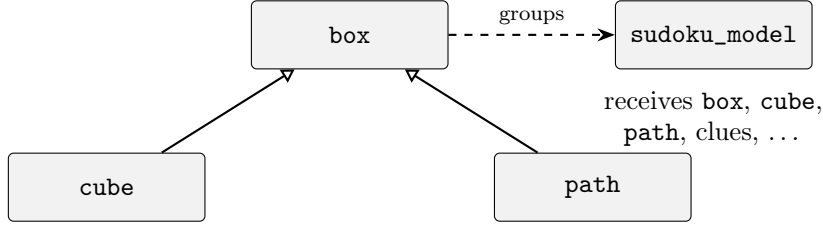


Figure 1: `cube` and `path` are subclasses of `box`. The model only knows about `box`.

```

class box:
    def __init__(self, n, cells=None):
        self.n, self.N = n, n * n
        self.cells = {}
        for (i, j), val in (cells or {}).items():
            self.add(i, j, val)

    def add(self, i, j, val=None):
        if not (0 <= i < self.N and 0 <= j < self.N):
            raise IndexError(f"cell ({i},{j}) is outside the grid")
        if val is not None:
            if not (1 <= val <= self.N):
                raise ValueError(f"value {val!r} is outside [1,{self.N}]")
            self.cells[(i, j)] = val
        else:
            self.cells.setdefault((i, j), None)

    def matrix(self): # zeros in free / non-member cells
        ...

```

(Simplified excerpt; the full version, with the type and conflict checks, is in the notebook.)

4.4 `cube` and `path` (R2, R3)

`cube(n,i,j)` adds the n^2 cells $(i n + p, j n + q)$. `path(n,start,end)` walks the straight segment between two cells, inclusive, in either direction. With (i_0, j_0) and (i_1, j_1) as the endpoints, the direction is given by the signs of the differences:

$$d_i = \text{sgn}(i_1 - i_0), \quad d_j = \text{sgn}(j_1 - j_0),$$

and the cells are $(i_0 + t d_i, j_0 + t d_j)$ for $t = 0.. \max(|i_1 - i_0|, |j_1 - j_0|)$. A segment that is neither horizontal nor vertical is rejected with `ValueError`.

```

class cube(box):
    def __init__(self, n, i, j):
        super().__init__(n)
        for a in range(n):

```

```

        for b in range(n):
            self.add(i * n + a, j * n + b)

class path(box):
    def __init__(self, n, start, end):
        super().__init__(n)
        (i0, j0), (i1, j1) = start, end
        if i0 != i1 and j0 != j1:
            raise ValueError("neither horizontal nor vertical")
        di = (i1 > i0) - (i1 < i0)
        dj = (j1 > j0) - (j1 < j0)
        for t in range(max(abs(i1 - i0), abs(j1 - j0)) + 1):
            self.add(i0 + t * di, j0 + t * dj)

```

4.5 Random clues (R4)

The function picks k distinct cells at random and pins each to a random value in $[1, N]$, by default with $k = n$. The result is a `box`, with no new class. The random generator is a parameter, so that tests are reproducible.

```

def random_clues(n, k=None, rng=None):
    rng = rng or random
    N, k = n * n, (n if k is None else k)
    g = box(n)
    all_cells = [(i, j) for i in range(N) for j in range(N)]
    for i, j in rng.sample(all_cells, k):
        g.add(i, j, rng.randint(1, N))
    return g

```

4.6 CSP model and solving (R5)

The model creates one integer variable in $[1, N]$ per cell. `add_groups` accepts an arbitrary number of groups and applies (1) and (2); `add_clues` applies only (2). The method `solve` returns the grid or `None`; the attribute `status` stores the solver status, which makes it possible to tell a puzzle with *no solution* (INFEASIBLE) apart from a *time limit reached* (UNKNOWN).

```

class sudoku_model:
    def __init__(self, n):
        self.n, self.N = n, n * n
        self.m = cp_model.CpModel()
        self.x = [[self.m.NewIntVar(1, self.N, f"x_{i}_{j}")
                    for j in range(self.N)] for i in range(self.N)]

    def add_groups(self, *groups):
        for g in groups:
            if len(g) > 1:
                self.m.AddAllDifferent([self.x[i][j] for (i, j) in g.cells])
            self._pin(g)

    def solve(self, time_limit=None):
        solver = cp_model.CpSolver()
        ...
        st = solver.Solve(self.m)
        self.status = solver.StatusName(st)
        if st in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            return [[solver.Value(v) for v in row] for row in self.x]
        return None

```

4.7 Full Sudoku (R6)

`full_sudoku` joins the N rows (left to right), the N columns (bottom to top, to exercise both directions of `path`), the N blocks and the clue group. It accepts extra groups, used by the variants in section 7.

Since the clues are random they may be contradictory. The strategy chosen in `generate_and_solve` is to **re-draw**: generate new clues until the puzzle has a solution (at most 100 attempts) and, if the attempts run out, return `None`. The justification is in section 6.

5 Validation

Validation is automatic (functions `validate` and `run_tests` in the notebook) and checks:

- that every row, column and $n \times n$ block of the solved grid contains exactly $\{1, \dots, N\}$;
- that the cells pinned by the clues keep their value in the solution;
- that `add` rejects coordinates outside the grid and values outside $[1, N]$, and that a value neither silently erases nor silently contradicts another;
- that `path` gives the same set of cells in both directions, works with a single point, and rejects diagonals;
- that a contradictory puzzle (two values 1 in the same row) returns `None` with status `INFEASIBLE`;
- the full flow (generate clues $\rightarrow$ assemble groups $\rightarrow$ solve $\rightarrow$ validate) with 20 different seeds for each of $n = 2$ and $n = 3$.

All tests passed. Nothing is hard-wired to 9×9 : the same code runs for $n = 2$ (4×4 grid) and up to $n = 6$.

6 Results

6.1 Random puzzles with no solution

To decide what to do when the random clues are contradictory, we measured the fraction of puzzles with no solution over 200 samples (seeds 0..199) for several n and k .

n	k (number of clues)	Puzzles with no solution
2	2	10 %
2	4	69 %
3	3	8 %
3	9	67 %
3	20	98 %

Table 2: Fraction of puzzles with no solution over 200 samples.

With $k = n$ (the default) the fraction is small, so re-drawing the clues almost always converges after one or two attempts. This is the reason for the chosen strategy. As k grows, each new clue may contradict the previous ones and the fraction rises quickly: in that regime re-drawing is no longer a good idea, and it would be better to generate the clues from a complete solution.

6.2 Scale

We measured the total time of `generate_and_solve` with $k = n$ clues (seed 1, a single run per value of n , wall-clock time).

n	Grid	Time (s)	Attempts
3	9×9	0.05	1
4	16×16	0.38	1
5	25×25	2.29	1
6	36×36	10.77	1

Table 3: Solving time as a function of n . All grids were validated.

The number of variables grows with $N^2 = n^4$ and the number of alldifferent constraints with $3N = 3n^2$, each over N variables, so the time grows fast but remains feasible up to $n = 6$. These times are for puzzles with very few clues, hence barely constrained: there are many solutions and the solver finds one quickly. We did not measure heavily constrained puzzles, nor repeat each measurement, so the values are indicative only. Re-running the notebook gives slightly different times.

7 Extensions

In every extension the CSP model stays **exactly the same**: groups are only added or replaced, which confirms the value of the `box` abstraction.

Diagonal Sudoku (X-Sudoku). Two `boxes`, one per main diagonal, passed through `extra`. The solution obtained was also validated on both diagonals.

Hyper-Sudoku / Windoku. Four `boxes` with 3×3 blocks with corners $(1, 1)$, $(1, 5)$, $(5, 1)$ and $(5, 5)$, overlapping the normal blocks. All four windows of the solution contain $\{1..9\}$.

Irregular Sudoku (jigsaw). The regular blocks are replaced by irregular regions, each a `box` built cell by cell from a letter map. It was tested on a 4×4 grid ($n = 2$) with four regions of 4 cells. Note: not every region layout admits a solution; the first layout we tried was impossible even without clues (the solver returned `INFEASIBLE`) and was replaced by one that has solutions.

Scale. See table 3.

The three-dimensional Sudoku extension ($n^2 \times n^2 \times n^2$) was **not implemented**.

8 Discussion and limitations

Design decisions.

- *CP-SAT instead of SCIP*: alldifferent is native and the propagation is specific to this constraint.
- *Two methods in the model* (`add_groups` and `add_clues`), for the reason in section 3.2. The assignment speaks of one method that imposes “all different” and pins the values; following it literally for the clue group would leave most puzzles with no solution.
- *Re-drawing clues* when the puzzle is impossible, because it is cheap with small k (table 2).
- *Result None plus the attribute status*, to tell “no solution” apart from “time limit reached”.

Limitations.

- Random clues do not guarantee a unique solution; the generated puzzle may have many solutions. The solver returns only one.
- Validation by sampling (20 seeds per n) is not a proof of correctness.
- The timing measurements are from a single run and for barely constrained puzzles.
- The 3D Sudoku was not done.

9 Conclusion

The `box` abstraction (a set of cells, some pinned, subject to “all different”) was enough to represent rows, columns, blocks, random clues and the diagonal, Windoku and jigsaw variants, without changing a line of the CSP model. Solving with CP-SAT is fast for the usual grids and remains feasible up to 36×36 with few clues. Still to do are the three-dimensional extension and a performance analysis with more heavily constrained puzzles.